AI FULL-STACK & DEPLOYMENT ENGINEER · TRACK 2.B · SESSION 6

Production AI APIs & Application Frameworks

From serving models to shipping products: streaming APIs, resilience patterns and structured outputs — and the first release of ClaimAssist, the application built across the next five sessions

SSE streaming · FastAPI async · Timeouts & retries · Idempotency keys · Pydantic outputs · OpenAI protocol · Streamlit vs Next.js

Track 2.A engineered the platform; Track 2.B builds one product on it — ClaimAssist v1 ships today.

Day 6 · 2 hours · concepts 45 min + lab 60 min + review 15 min

SESSION 6 · AGENDA

Session agenda

45 min

Concepts & code walkthrough

The ClaimAssist product and its five-session roadmap; sync vs streaming and time-to-first-token; SSE mechanics; async FastAPI; timeouts, retries and idempotency keys; structured outputs with Pydantic; the OpenAI-compatible protocol; Streamlit vs Next.js.

60 min

Lab — ClaimAssist v1

docker compose brings up the OpenAI-compatible model server and the ClaimAssist API. Speak the protocol raw with curl, stream over SSE, break the upstream mid-stream, prove idempotency, then chat through Streamlit.

15 min

Review & wrap-up

Compare failure-handling evidence across pairs, map each lab mechanism to production, and preview Session 7: LiteLLM routing and the Next.js streaming UI.

Objective: ship ClaimAssist v1 — a streaming, resilient, structured production API over the local model, with a working chat front end.

THE PRODUCT

ClaimAssist: an insurance claims-status copilot

The business problem

- **Insurance call centres are flooded with claim-status queries — “where is my claim?”, “why was it rejected?”, “when will I be paid?”**
- High volume, low complexity: the answer already exists in the claim record and the policy documents
- Every deflected call is measurable saving; every wrong answer is a compliance risk — the application must be grounded, governed and observable
- The target: a copilot that answers status questions accurately, cites policy clauses, and refuses what it cannot support

What is built, session by session

v1	**Today —** Production API: streaming, resilience, structured outputs + Streamlit chat
v2	**S7 —** LiteLLM routing across providers + Next.js streaming web UI
v3	**S8 —** RAG over the policy documents with clause citations + multimodal intake
v4	**S9 —** MCP tool-use: claim lookup, policy search, drafted (never sent) emails
v5	**S10 —** LLMOps: versioned prompts, Langfuse tracing, guardrails, PII redaction

Why this product: ClaimAssist mirrors Capstone #1 (Insurance · Claims Status Copilot). Every pattern built in Sessions 6-10 transfers directly into the capstone deliverable.

THE PRODUCT

Product evolution: v0 (a served model) → v1 (a product)

v0 — Track 2.A end state

Qwen model container
`POST /generate  :8080`

curl / promptfoo
the only clients

A served model — engineered, gated, monitored. Not yet a product.

today

v1 — ClaimAssist ships today

Streamlit chat
`:8501 · host`
st.chat_input · streams tokens

ClaimAssist API
`:8000 · FastAPI`
/claims · /ask · /ask/stream
retries · idempotency · Pydantic

llm-server:openai
`:8090 · container`
/v1/chat/completions
non-stream + SSE

`data/claims.json` six mock records ground every answer (mounted read-only)

UI → API → model. The UI never calls the model directly.

The road ahead:

- v2 · S7 LiteLLM + Next.js
- v3 · S8 RAG + citations
- v4 · S9 MCP tool-use
- v5 · S10 LLMOps + guardrails

Layering rule: product logic — grounding, structure, resilience, idempotency — lives in the API layer. The model serves; the API is the product.

PRODUCTION API PATTERNS

Sync vs streaming: time-to-first-token is the UX metric

Synchronous `POST /ask — one response, at the end`

user waits — a blank screen for the full generation | **answer**

`t = 0` — `t ≈ 20-60 s: everything at once`

Streaming `POST /ask/stream — tokens as they are produced (SSE)`

first | | | | | | | | | | | | | |

`t ≈ 1 s: first token — the user is reading while the model is still writing` — `same total time`

Time-to-first-token (TTFT) is the user-perceived latency of an LLM product. Total generation time barely changes between the two designs; perceived responsiveness changes by an order of magnitude. The rule: user-facing generation streams. Synchronous calls remain correct for machine-to-machine consumers that need the complete, validated text — ClaimAssist ships both endpoints for exactly this reason.

PRODUCTION API PATTERNS

Server-Sent Events: the wire format under every LLM chat

```
$ curl -N localhost:8090/v1/chat/completions \
    -d '{"model":"qwen-local", "stream": true,
         "messages":[...]}'

HTTP/1.1 200 OK
Content-Type: text/event-stream

data: {"object":"chat.completion.chunk","choices":
  [{"delta":{"role":"assistant","content":""}}]}

data: {"choices":[{"delta":{"content":"Your "}}]}

data: {"choices":[{"delta":{"content":"claim "}}]}

data: {"choices":[{"delta":{"content":"is "}}]}

data: {"choices":[{"delta":{},
        "finish_reason":"stop"}]}

data: [DONE]
```

Reading the format

- One HTTP response that stays open; each event is a “data: ” line terminated by a blank line
- The OpenAI dialect: JSON chunks carrying choices[0].delta.content — concatenate the deltas to rebuild the answer
- “data: [DONE]” is the termination sentinel — clients stop reading there
- curl -N disables client buffering; servers send Cache-Control: no-cache and disable proxy buffering

In today's lab

The model server emits this exact stream (model_server/app.py); the ClaimAssist API consumes it and re-emits simplified data: {"delta": …} events to its own clients — a server-side transformation you will trace end to end in Step 3.

PRODUCTION API PATTERNS

Why SSE, not WebSockets, for LLM chat

Server-Sent Events

- **One-directional: server → client — precisely the shape of "question up, answer streams down"**
- Plain HTTP response: works through proxies, load balancers and corporate networks unchanged
- Stateless per request: any replica can serve the next question — no connection affinity
- Trivial clients: curl -N, fetch(), EventSource — and every OpenAI SDK

WebSockets

- **Bidirectional, persistent connection — a protocol upgrade out of plain HTTP**
- Stateful: load balancers must pin connections; reconnection and resume logic is yours to write
- Message framing, heartbeats and auth-on-upgrade are all custom protocol work
- Earns its cost when both sides genuinely push: collaborative editing, multiplayer state, live voice

Fit over power: chat completion is request → response-stream, and SSE is that shape with no residual machinery. OpenAI and Anthropic both stream over SSE — which is why every SDK, proxy and framework in this course speaks it natively.

PRODUCTION API PATTERNS

Async in FastAPI: the event loop and slow upstreams

- **FastAPI executes async def handlers on a single event loop; await releases the loop while I/O is in flight**
- An LLM call is extreme upstream I/O: seconds of waiting per request — the handler is idle almost its entire lifetime
- Async pays off exactly here: many concurrent requests, each waiting on a slow upstream, overlapped by one process
- The corollary: one BLOCKING call inside an async handler (requests.post, time.sleep) freezes the loop — every in-flight request stalls, not just the offending one
- Rule: in async handlers, only async I/O — httpx.AsyncClient, asyncio.sleep

```
# api/app.py — async end to end
@app.post("/ask", response_model=AskResponse)
async def ask(req: AskRequest, ...):
    answer = await call_llm(...)
    # ^ releases the event loop while
    #   the model is generating

# inside call_llm — async client, never `requests`:
async with httpx.AsyncClient(
        timeout=REQUEST_TIMEOUT) as client:
    r = await client.post(
        f"{LLM_BASE_URL}/chat/completions", ...)

# backoff between retries — asyncio, not time:
await asyncio.sleep(BACKOFF_BASE_S * 2**attempt)
```

When async does NOT matter

CPU-bound work (the model server's own generation loop) gains nothing from async — it needs processes or GPUs, not an event loop. Async is a concurrency tool for waiting, and the API layer is almost all waiting.

RESILIENCE PATTERNS

Timeouts and bounded retries with backoff

```
# api/app.py — every upstream LLM call goes through this
REQUEST_TIMEOUT = httpx.Timeout(90.0, connect=5.0)
MAX_RETRIES     = 2      # 1 attempt + 2 retries
BACKOFF_BASE_S  = 0.5    # 0.5 s, then 1.0 s

for attempt in range(MAX_RETRIES + 1):
    try:
        async with httpx.AsyncClient(
                timeout=REQUEST_TIMEOUT) as client:
            r = await client.post(
                f"{LLM_BASE_URL}/chat/completions",
                json=payload, headers=headers)
            r.raise_for_status()
            return r.json()["choices"][0]
                        ["message"]["content"]
    except (httpx.TimeoutException,
            httpx.HTTPError) as exc:
        last_error = exc
        if attempt < MAX_RETRIES:
            await asyncio.sleep(
                BACKOFF_BASE_S * (2 ** attempt))

raise HTTPException(status_code=503, detail=...)
```

The three disciplines

- **Timeout, always: the default is an infinite hang. Split the budget — connect tight (5 s), read generous (90 s), because generation is legitimately slow.**
- Bounded retries: three attempts, then a loud 503 with the cause. Unbounded retries amplify incidents.
- Exponential backoff: 0.5 s, 1.0 s — never hammer a sick upstream at full rate.

Lab Step 3

You will kill the model container mid-request and watch this exact loop engage: the backoff pauses, then the bounded failure. Resilience is observed, not asserted.

RESILIENCE PATTERNS

Idempotency keys: retried POSTs must not execute twice

```
# api/app.py — the Idempotency-Key pattern on POST /ask
@app.post("/ask", response_model=AskResponse)
async def ask(req: AskRequest,
              idempotency_key: str | None =
                  Header(None, alias="Idempotency-Key")):
    if idempotency_key:
        cached = idempotency_get(idempotency_key)
        if cached is not None:
            return AskResponse(**cached)
            # replay: identical request_id,
            # zero LLM calls, zero cost

    answer = await call_llm(...)      # the real work
    ...
    if idempotency_key:
        idempotency_put(idempotency_key,
                        response.model_dump())

# lab: in-memory dict with a 600 s TTL
# production: Redis  SET key NX EX 600  (shared)
```

Why this exists

- **A timeout is ambiguous: the client cannot know whether the server did the work. The safe client retries — so the server must make retries harmless.**
- The client owns the key (one per logical operation); the server caches the first outcome and replays it.
- For ClaimAssist the double-execution cost is an LLM bill; for a payments API it is a double charge — same pattern, Stripe made the header canonical.
- GET is naturally idempotent; POST needs this machinery.

Lab Step 4 proof: the same POST /ask sent twice with the same Idempotency-Key returns byte-identical responses — the request_id matches, and the second reply is instant because no LLM call ran.

PRODUCTION API PATTERNS

Structured outputs: Pydantic models, not free text

```python
# api/app.py — the /ask response contract
class AskResponse(BaseModel):
    answer: str
    claim_id: str | None = None
    status: str | None = None
    confidence: Literal["high",
                        "medium", "low"]
    request_id: str

@app.post("/ask", response_model=AskResponse)
# FastAPI validates every response on the
# way OUT — an invalid shape cannot ship
```

```
// what the wire actually carries
{ "answer": "Your claim CLM-1002 is approved;
   payment is processing (3-5 working days).",
  "claim_id": "CLM-1002", "status": "Approved",
  "confidence": "high", "request_id": "req-8a41…" }
```

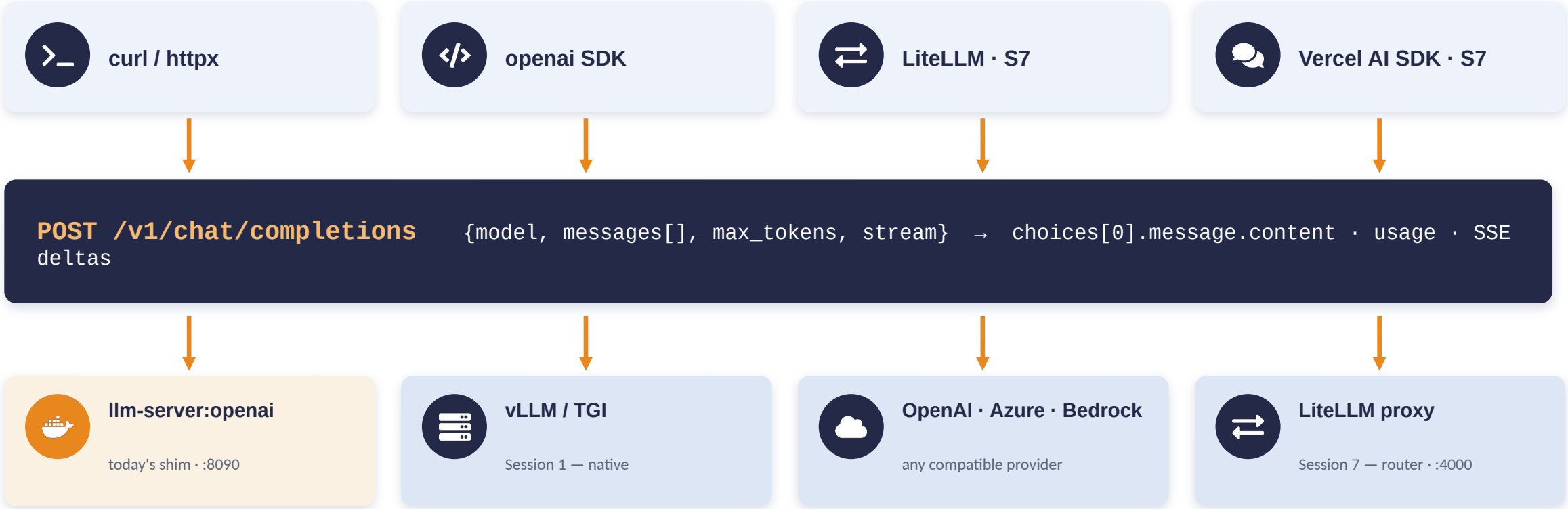
Why structured beats free text

- **Product features consume FIELDS: the UI badges confidence, routing logic filters on status, analytics counts by claim_id**
- Free text forces regex parsing downstream — brittle against every prompt or model change
- response_model is an enforced, self-documenting contract: OpenAPI schema for consumers, validation on every response
- Literal["high","medium","low"] makes invalid confidence values unrepresentable
- v1 keeps the LLM's prose inside `answer` and computes the structured fields deterministically — the model is never trusted to emit the envelope

Session 8: same contract, better semantics — confidence derived from retrieval distance, citations added beside it.

INTEGRATION STANDARD

The OpenAI-compatible protocol: one wire format, every tool



Today's shim (model_server/app.py): the lab wraps the Qwen pipeline in this protocol — non-stream JSON with tokenizer-counted usage, and SSE chunks ending in [DONE]. It generates the full text first and streams it out in chunks (simple and reliable on CPU); vLLM streams true token-by-token. The wire format is identical — which is the entire point.

INTEGRATION STANDARD

The local/cloud switch: three lines of configuration

CLOUD PATH · OPTIONAL

```
# .env — LOCAL path (default)
LLM_BASE_URL=http://model:8090/v1
LLM_API_KEY=local
LLM_MODEL=qwen-local

# free, offline, private —
# the whole lab runs on this
```

```
# .env — CLOUD path (optional)
LLM_BASE_URL=https://api.openai.com/v1
LLM_API_KEY=sk-your-key-here
LLM_MODEL=gpt-4o-mini

docker compose up -d api   # recreate api
# every endpoint behaves identically
```

What changes and what does not

Changes: answer quality, latency profile, per-token cost, and streaming becomes true token-by-token. Does not change: one line of application code, the endpoints, the response contract, the SSE framing, the Streamlit UI. Any OpenAI-compatible provider works — OpenAI, Azure OpenAI, a hosted vLLM, a colleague's machine. Keys stay in .env, never in code or images.

Session 7: the static .env edit becomes a LiteLLM proxy — routing, fallbacks and cost policy across local and cloud models, behind the same protocol.

APPLICATION FRAMEWORKS

Streamlit vs Next.js: an honest comparison

	Streamlit (today's lab)	Next.js + Vercel AI SDK (Session 7)
Development speed	A working streaming chat in ~90 lines of Python, one afternoon	Scaffold, components, API routes — days; a real front-end codebase
Customisation & branding	Theme knobs; layouts constrained by the framework	Total control — it is your front end, pixel by pixel
Auth · SEO · routing	Bolt-on and awkward; no real routing or SEO story	First-class: middleware auth, server rendering, file-based routing
Streaming UX control	st.write_stream renders deltas — sufficient, not tunable	Per-token control via useChat/streamText; optimistic UI, cancellation
Operations	One Python process; fine behind internal SSO	Node build, CDN/edge deployment — standard web operations
Best fit	Internal tools, analytics teams, demos, fast iteration	Customer-facing production applications

Choose by audience, not by preference: internal claims-ops dashboard → Streamlit. Public customer portal → Next.js. ClaimAssist builds both, against the same API.

HANDS-ON LAB · 60 MINUTES · RUNS FULLY LOCALLY

Ship ClaimAssist v1

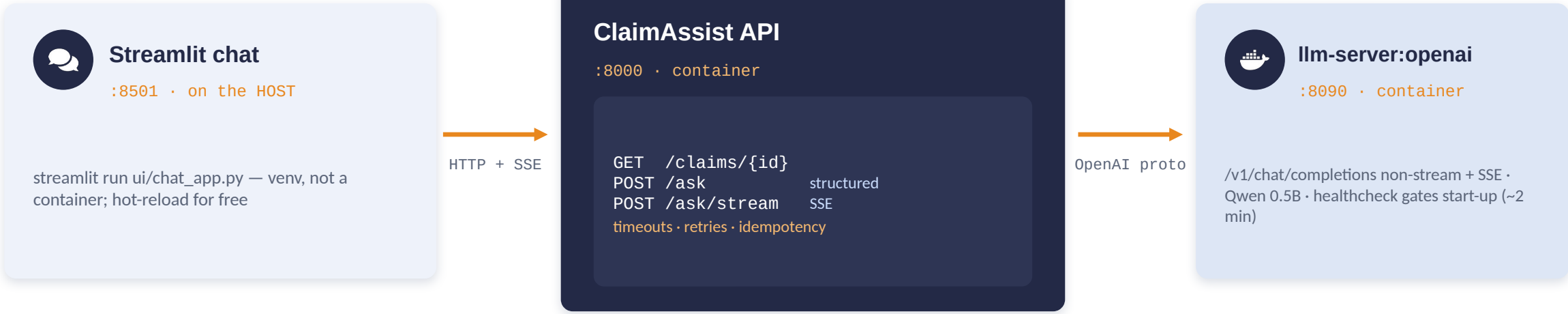
A streaming, resilient, structured production API — with a chat front end

docker compose brings up the OpenAI-compatible model server (:8090) and the ClaimAssist API (:8000). You will speak the protocol raw with curl, stream over SSE, kill the model mid-stream and watch the retry and in-band error handling, prove idempotency, and chat through Streamlit (:8501).

Step 0 setup · Step 1 stack + raw protocol · Step 2 the API · Step 3 SSE failure drill · Step 4 idempotency · Step 5 Streamlit · Step 6 framework discussion

LAB · ARCHITECTURE

The v1 stack: two containers plus a host UI



Health-gated start-up

The API declares depends_on: model: condition: service_healthy — it starts only after the model's HEALTHCHECK passes. The local equivalent of a readiness probe plus rollout ordering.

Configuration flow

.env → api container only (LLM_BASE_URL / LLM_API_KEY / LLM_MODEL). data/claims.json is mounted read-only. The UI holds no LLM configuration at all — it knows only the API's address.

Why the UI is not in compose: the two containers are the deployable product; the Streamlit app is a development-speed front end. Containerising it adds build friction and removes hot-reload while teaching nothing — Session 7 introduces the production front end properly.

LAB · STEP 0

Setup: environment and UI venv (5 min)

```
cd session6_lab

# .env — the local/cloud switch (Step 13's slide)
echo 'LLM_BASE_URL=http://model:8090/v1' >  .env
echo 'LLM_API_KEY=local'                 >> .env
echo 'LLM_MODEL=qwen-local'              >> .env

# venv for the Streamlit UI (host-side)
python3 -m venv .venv
source .venv/bin/activate
#   Windows: .venv\Scripts\activate
pip install -r ui/requirements.txt

docker --version && docker compose version
```

Lab materials (from the handout)

session6_lab contains: model_server/ (the OpenAI shim + Dockerfile), api/ (ClaimAssist API), ui/chat_app.py, data/claims.json + data/policies/, docker-compose.yml, .env.example and the README with these exact steps.

Verification notes

- **Address subtlety, settled now: inside compose the API reaches the model at http://model:8090/v1 (service DNS). From your host shell the same server is http://localhost:8090 — Step 1 curls it that way.**
- The .env configures only the api container. The UI reads nothing from it — it knows only the API's address.
- The venv is only for Streamlit and requests; the containers carry their own pinned dependencies.
- Docker and Compose carry over from Track 2.A unchanged; roughly 4 GB free RAM suffices — one model container this time.

LAB · STEP 1

Start the stack and speak the protocol raw (10 min)

```
docker compose up -d --build
docker compose ps    # both (healthy) — model ~2 min

# the OpenAI wire format, raw — no SDK
curl -s localhost:8090/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen-local",
       "messages":[{"role":"user","content":
         "What is an insurance claim?"}],
       "max_tokens":60}' | python3 -m json.tool
# → choices[0].message.content + usage tokens

# now streaming — watch the SSE events arrive
curl -N localhost:8090/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen-local","stream":true,
       "messages":[...],"max_tokens":60}'
# data: {...delta...}  ...  data: [DONE]
```

Observe and interpret

- **The non-streaming JSON is the industry-standard shape: id, choices[0].message.content, finish_reason, and usage counted with the real tokenizer — what providers bill on.**
- In the streaming output, count the events: role chunk first, then one delta per word, then finish_reason: "stop", then [DONE] — the exact frames from the theory block.
- -N turns off curl's buffering; without it the whole stream appears at once and the demonstration is lost.
- This is the model server's own port (8090). From Step 2 onward you use only the ClaimAssist API (8000) — clients of a product never talk to the model directly.

LAB · STEP 2

Explore the ClaimAssist API: lookup and structured /ask (10 min)

```
# raw record lookup
curl -s localhost:8000/claims/CLM-1002 \
     | python3 -m json.tool

# 404 handling — a contract, not a crash
curl -s -i localhost:8000/claims/CLM-9999 | head -1
# HTTP/1.1 404 Not Found

# structured, grounded question answering
curl -s localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question":"What is the status of my claim
        and when will I be paid?",
       "claim_id":"CLM-1002"}' | python3 -m json.tool

# {"answer": "...", "claim_id": "CLM-1002",
#  "status": "Approved", "confidence": "high",
#  "request_id": "req-..."}
```

Observe and interpret

- **The /ask response is the AskResponse contract from the theory block — five typed fields, validated by FastAPI on the way out.**
- confidence is "high" because the answer is grounded in an injected claim record; ask without claim_id and it drops to "low" — the v1 heuristic, honest about what it knows.
- Grounding is inspectable: the system prompt contains the CLM-1002 record verbatim (see build_messages in api/app.py). Check the model's answer against the record you fetched one command earlier.
- Open localhost:8000/docs — the OpenAPI schema shows the response model to any consumer.

LAB · STEP 3

SSE deep-dive: kill the upstream mid-stream (10 min)

```
curl -N localhost:8000/ask/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"Why was my claim rejected and
       what can I do now?","claim_id":"CLM-1003"}'
# data: {"delta": "Your "}  ...  data: [DONE]

# run it again — and in a SECOND terminal:
docker kill claimassist-model

# outcome A (killed before first chunk):
#   pause 0.5 s ... pause 1.0 s ... then:
# data: {"error": "LLM upstream unavailable
#        after 3 attempts: ConnectError"}
# outcome B (killed mid-stream): error event
#   immediately — no retry, no repeated text

docker compose up -d model   # recover
docker compose ps            # wait (healthy), re-run
```

Observe and interpret

- **A streaming response has already committed 200 OK — failure MUST travel in-band as an event. Clients that ignore error events hang forever.**
- The retry asymmetry is deliberate: safe to retry before the first byte, unsafe after — replaying a half-sent stream would duplicate text.
- The backoff pauses you feel (0.5 s, 1.0 s) are the code from the resilience slide executing.

Deliverable 3: the transcript showing the in-band error event (and the backoff pause if you hit outcome A), plus the successful re-run after recovery.

LAB · STEP 4

Idempotency: the same POST twice, one execution (10 min)

```
KEY="demo-key-001"

curl -s localhost:8000/ask \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $KEY" \
  -d '{"question":"Where is my claim?",
       "claim_id":"CLM-1005"}' | python3 -m json.tool
# ... "request_id": "req-3f6a2b81c04d"

# the EXACT same command again
curl -s localhost:8000/ask \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $KEY" \
  -d '{"question":"Where is my claim?",
       "claim_id":"CLM-1005"}' | python3 -m json.tool
# instant  ... "request_id": "req-3f6a2b81c04d"
# IDENTICAL id → cached replay, no 2nd LLM call
```

Observe and interpret

- **The matching request_id is the proof: the handler returned the stored response before any work ran. The instant latency is the corroborating evidence.**
- Control experiment: change the key or drop the header — a new request_id appears, and the answer text may differ (the model is sampled).
- The lab cache is a process-local dict with a 600 s TTL; the api/app.py comment marks the production substitution: Redis SET NX EX, shared across replicas.

Deliverable 4: both responses side by side with the same Idempotency-Key, request_id highlighted and identical.

LAB · STEP 5

The Streamlit chat: the product face of v1 (10 min)

```
source .venv/bin/activate    # if not active
streamlit run ui/chat_app.py
# → http://localhost:8501

# sidebar: pick a claim → chat streams via /ask/stream
```

Conversations to run

- CLM-1005 → “Where is my claim?” — grounded status answer, streamed token by token
- CLM-1003 → “Why was my claim rejected?” — the record's notes cite clause M-2.3 (licence expired on the accident date)
- CLM-1003 → “What exactly does clause M-2.3 say?” — v1 cannot answer well: it sees the claim record, not the policy documents
- CLM-1002 → “Why was the approved amount lower than claimed?” — the notes name H-4.2; the clause text itself is out of reach

Observe and interpret

- **The whole UI is ~90 lines of Python (ui/chat_app.py): st.chat_message, st.chat_input, st.write_stream over the API's SSE — the Streamlit value proposition made concrete.**
- The UI holds no LLM configuration and never touches port 8090 — layering enforced by construction.
- Kill the API while chatting: the UI shows a handled error message, not a crash — the in-band error path from Step 3, surfacing in the front end.
- The policy-clause gap is the designed cliff-hanger: answers cite clause ids they cannot quote. Session 8 adds retrieval over data/policies/ with citations.

Deliverable 5: a screenshot of the chat answering “Why was CLM-1003 rejected?” with the streamed response visible.

LAB · STEP 6

Framework discussion: what did Streamlit buy, and cost? (5 min)

What Streamlit bought today

- A working streaming chat UI in ~90 lines of Python — no JavaScript, no build step, no front-end expertise
- st.write_stream consumed the SSE endpoint directly; session state gave chat history for free
- Ideal delivery speed for internal tools, analytics teams and stakeholder demos — this UI was effectively free

What it would cost at product scale

- Branding and layout hit the framework's walls quickly; a customer portal cannot look like a data app
- No first-class auth, routing or SEO — bolting them on fights the framework
- Streaming UX is take-it-or-leave-it: no per-token control, optimistic UI or cancellation affordances
- Shipping it customer-facing is pitfall #6 on the next-but-one slide — internal speed misapplied to a public product

Deliverable 6 (written): three sentences — when is Streamlit the right choice, when is it the wrong one, and what specifically does Session 7's Next.js + Vercel AI SDK stack add? The defensible answer chooses by audience, not by preference.

SYNTHESIS

Mapping the local lab to production

Local mechanism — today's lab	Production counterpart
`llm-server:openai — full text, then chunked SSE`	vLLM / TGI (true token streaming) or a managed provider endpoint
`LLM_BASE_URL / KEY / MODEL in .env`	Provider configuration; LiteLLM proxy with routing + fallbacks (S7)
`Manual httpx retry loop with backoff`	SDK max_retries / service-mesh retry policy — the same contract
`In-memory Idempotency-Key dict (TTL 600 s)`	Redis SET key NX EX — shared across replicas, survives restarts
`compose healthcheck + depends_on: service_healthy`	Kubernetes liveness/readiness probes + rollout ordering
`SSE via FastAPI StreamingResponse`	The same protocol through a gateway/CDN with buffering disabled
`Streamlit on the host (:8501)`	Internal tool behind SSO; Next.js for the customer product (S7)

Because the application speaks the OpenAI-compatible protocol, every substitution in the right column happens behind an unchanged interface.

LAB DELIVERABLES

Evidence to submit at the end of the session

1 Wire-format evidence

Transcripts of the non-streaming and streaming curls against :8090/v1/chat/completions — the raw OpenAI JSON and the SSE events ending in [DONE] (Step 1).

2 Structured output

The /ask response for CLM-1002 showing all five fields of the AskResponse contract, including confidence (Step 2).

3 Failure-handling evidence

The /ask/stream transcript with the in-band error event after the model container was killed, plus the successful re-run after recovery (Step 3).

4 Idempotency proof

Both /ask responses with the same Idempotency-Key, request_id identical across the two (Step 4).

5 UI evidence

A screenshot of the Streamlit chat answering “Why was CLM-1003 rejected?” with the streamed answer visible (Step 5).

6 Written answer

Three sentences: when Streamlit is the right choice, when it is the wrong one, and what Session 7's stack adds (Step 6).

Keep the session6_lab directory — Sessions 7-10 evolve this same application.

PITFALLS

Common failure modes in production AI APIs

Blocking calls in async handlers

One requests.post or time.sleep inside an async def freezes the event loop — every in-flight request stalls, not just the offender. Async handlers use httpx.AsyncClient and asyncio.sleep exclusively.

No timeout on upstream LLM calls

The default is an infinite hang; a single sick upstream exhausts the worker pool. Every call carries an explicit budget — tight connect, generous read — and fails loudly.

Retrying non-idempotent POSTs without keys

A timed-out POST retried blindly executes twice: two LLM bills, two tickets, two charges. Retries are enabled only alongside the Idempotency-Key pattern.

Parsing free-text LLM output with regex

Extracting fields from prose breaks on every prompt or model change. Define a Pydantic contract, keep prose inside `answer`, and compute structured fields deterministically.

Streaming without client failure handling

An SSE response has already sent 200 OK — failures arrive as in-band events, and clients that ignore them hang forever. Emit and handle error events; never retry mid-stream.

Shipping Streamlit as the customer product

Internal delivery speed misapplied: no real auth, routing, SEO or branding control. Streamlit serves internal users; the customer front end is a web framework (Session 7).

SESSION 6 · RECAP

Capabilities established today

✓ **Ship a product API**	ClaimAssist v1: grounded /ask, SSE /ask/stream, claim lookup with contract 404s.
✓ **Stream over SSE**	Time-to-first-token as the UX metric; data: events ending in [DONE]; in-band errors.
✓ **Engineer resilience**	Explicit timeouts, bounded retries with backoff, and Idempotency-Key replay on POST.
✓ **Structure outputs**	A Pydantic response contract — fields for product features, validated on the way out.
✓ **Speak the standard**	The OpenAI-compatible protocol: local container and cloud provider behind one interface.

Next session

ClaimAssist v2 — LiteLLM Routing & the Next.js Streaming UI

A LiteLLM proxy replaces the static .env switch — routing, fallbacks and retries across local and cloud models — and a Next.js app with the Vercel AI SDK becomes the customer-facing streaming front end, against the same API built today.

Retain the session6_lab stack — Session 7 routes this same application.